

综合题包括：系统框图，流程图，程序

可以带尺子？带计算器

1 数电部分

1.1 数码制

数字量：变化在时间上和数量上都是不连续的，存在一个最小数量单位△

模拟量：数字量以外的物理量

2与16进制转化，每四位为一组，从右向左组合，不够的补0

二进制的加减乘除都可以用移位来解决：

二进制加法：

1001 (9)

+1011 (11)

10100 (20)

二进制减法：减去一个数相当于加上他的补码

二进制乘法：乘2就是左移1位，乘4就是左移2位，除法原理相同。

补码：符号位0+1-，除符号位都取反+1

1.2 逻辑运算

与

- 条件同时具备，结果发生
- $Y=A \text{ AND } B = A \& B = A \cdot B = AB$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

A

B

&

Y

与

A

B

Y

或

- 条件之一具备，结果发生
- $Y=A \text{ OR } B = A+B$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

A

B

≥1

Y

或

A

B

Y

非

- 条件不具备，结果发生
- $Y=A' = \text{NOT } A$

A	Y
0	1
1	0

A

1

Y

非

A

Y

图案也记住

与非 或非 记下（与可以用与非+非来实现）

几种常用的复合逻辑运算

与非

A

B

Y

A

B

&

Y

与非

$Y=(A \cdot B)'$

或非

A

B

Y

A

B

≥1

Y

或非

$Y=(A+B)'$

与或非

A

B

C

D

Y

A

B

&

C

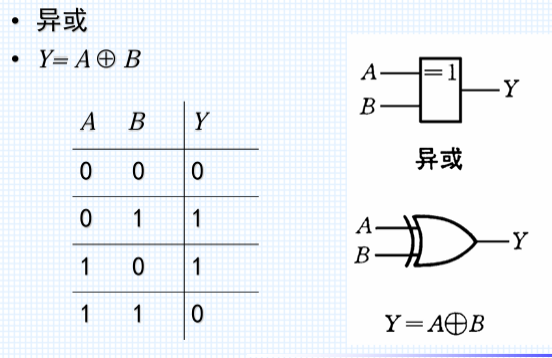
D

≥1

Y

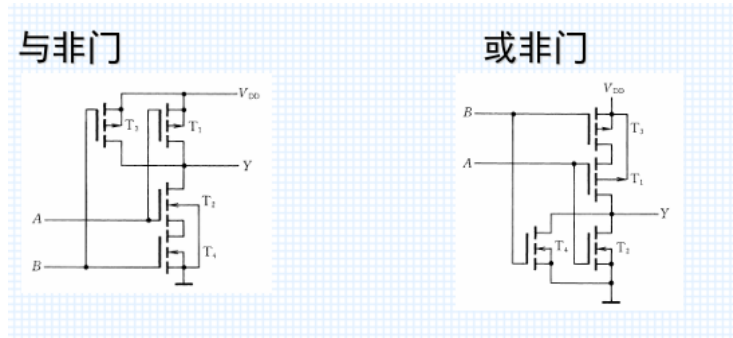
与或非

$Y=(A \cdot B + C \cdot D)'$



异或是加法基础

门电路：用半导体器件可以构建有逻辑功能的电路



会判断就行

有的芯片是多个与非门的集成，组合多个管脚

1.3 组合逻辑电路

1.3.1 概念

数字电路是由简单的门构成复杂的功能，组合电路由门电路组成，不含记忆（存储）元件；其输出只与该时刻的输入有关。

时序逻辑电路包含记忆元件，其输出不仅取决于当前时刻输入，也和电路原来的状态有关

非时序逻辑电路就是组合逻辑电路

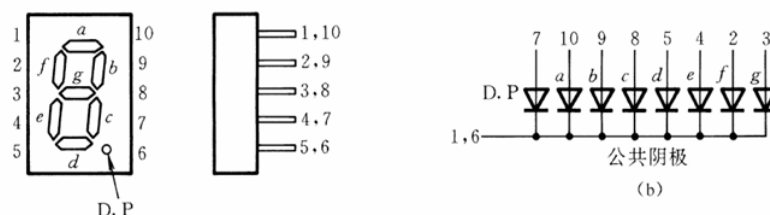
1.3.2 举例

只要会举例，能说出名字，知道大概实现什么功能，具体结构与功能不用管

编码器：把一堆输入转化为二进制编码

译码器：把一堆二进制编码转化为输出

八段数码管（必考）：共阴：亮对应1对应高电平；共阳：亮对应0对应低电平



按照a-h，从右到左编码

加法器：1位加法器（半加器），全加器，只需知道用一堆与非逻辑门可以搭出来一个加法器即可

串行加法器：按照从低到高一位一位加上去，结构简单但是慢

并行加法器：第*i*位的进位是输入信号两个加数第*i*位之前各位(0~*j*-1)的函数，可以在相加前由AB两数确定。结构复杂但是快，每一位都基本同时产生。

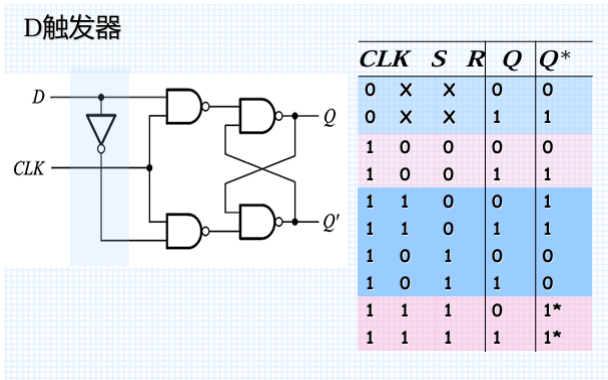
(不用记住结构，只知道有这么两个东西存在即可)

1.4 时序逻辑电路

记忆单元：触发器：一个逻辑门的输出是另一个的输入

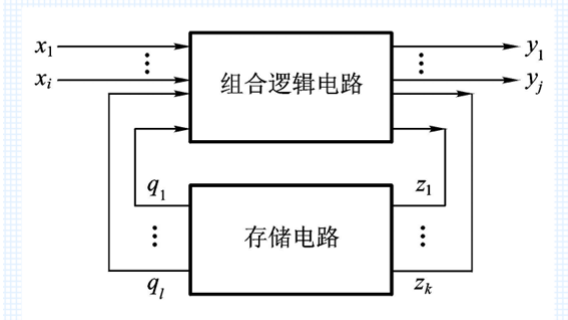
(记住一个就行，没怎么考过)

D触发器：



Q是下一时刻的输出，Q*是上一时刻输出

二、时序电路的一般结构形式与功能描述方法



如果是简答题考到就画这张图

1.4.1 举例

寄存器：寄存一组二值代码

移位寄存器：存储加移位

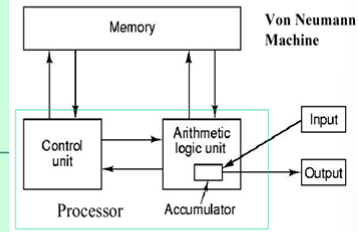
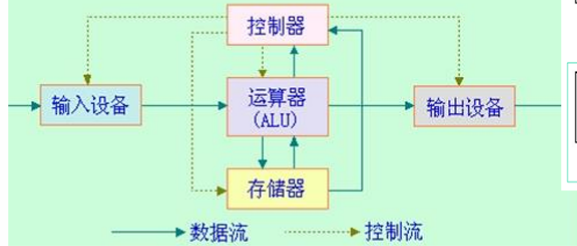
同步/异步计数器：计数的

只要能说出名字，知道是干什么的，无需记结构

2 8051部分

冯诺依曼体系结构

冯·诺依曼计算机结构



可能根据具体问题，问你这几个模块怎么工作的。

2.1 汇编Assembly language

只考判断选择

由什么构成：

[label]:Mnemonic [operands] [;comment]

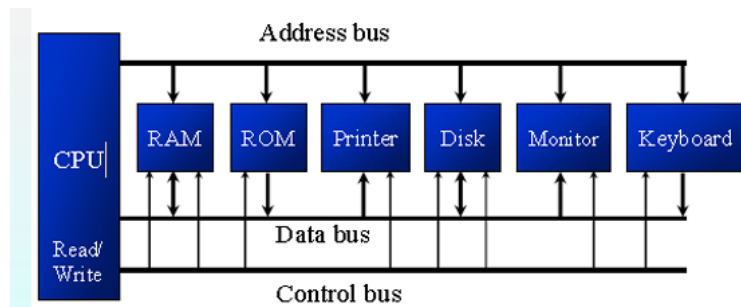
标号：助记符 操作数 ;注释

MOV移动；ADD相加；SJMP跳转

2.2 单片机CPU三总线结构

bus: The CPU is connected to memory and I/O through strips of wire called a bus. Carries information from place to place

有三条总线：Address bus, Data bus, Control bus



2.3 PSW

ALU: 完成算术运算、逻辑、位运算等等，相关寄存器有A B PSW

A(ACC): For all arithmetic and logic instructions (算术与逻辑)

B: For multiplication and division (乘法)

PSW: also referred to as the flag register, is an 8 bit register. Only 6 bits are used (状态寄存器)

PSW

D7	D6	D5	D4	D3	D2	D1	D0
Cy	AC	F0	RS1	RS0	OV	—	P

Cy (Carry): Carry flag

AC (Auxiliary Carry): Auxiliary carry flag

F0 (Flag): Available to the user for general purpose

RS1、RS0: Register Bank selector

OV (Overflow) : Overflow flag

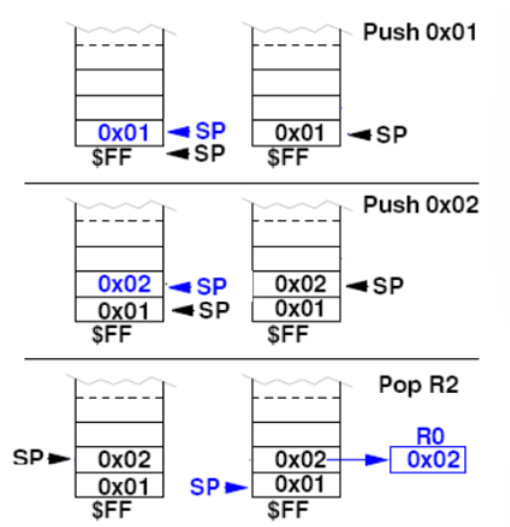
P (Parity): Parity flag. Set/cleared by hardware each instruction cycle to indicate an odd/even number of 1 bits in the accumulator.

每一位分别是什么

2.4 Stack堆栈

只考选择题

8051中堆栈指针指向最后一个被用掉的值，当push的时候先指针加一再写入数据；当pop的时候先将数据读出去，再写入数据



2.5 PC

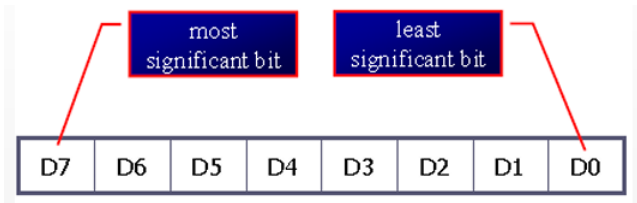
PC (program counter) points to the address of the next instruction to be executed (指向下一条被执行指令的位置)

当遇到中断响应时，将当前PC值压入堆栈，PC改到中断响应程序处，执行完后PC值出栈继续运行

2.5 register

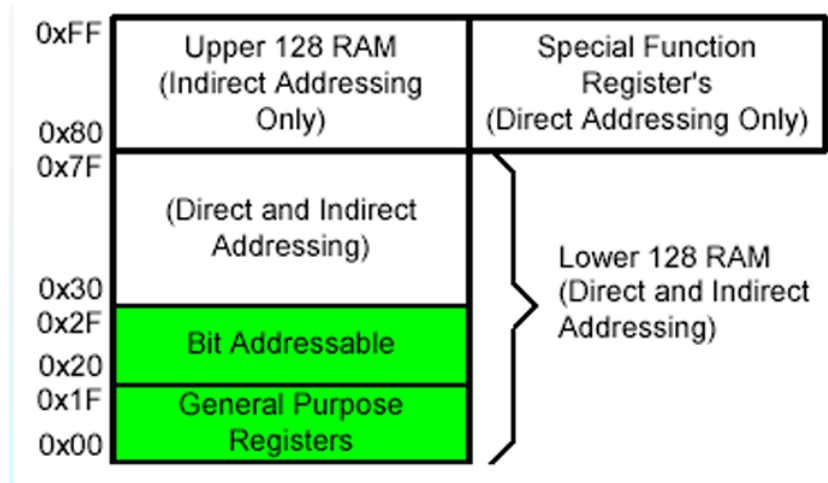
Register are used to store information temporarily, while the information could be: a byte of data to be processed, or an address pointing to the data to be fetched (数据或者是指向可获取的数据的地址)

The vast majority of 8051 register are 8-bit registers There is only one data type, 8 bits, any data larger than 8 bits must be broken into 8-bit chunks before it is processed (大部分是八位, 多余八位的要分为八位块)



右边是低位，左边是高位

Internal RAM (选择题概率高)



0x00-0x1F: 工作寄存器区

0x20-0x2F: 位寄存器，有单独编号，不会考这么细

0x30-0x7F: 用户可直接用的RAM

0x80-0xFF: 特殊功能寄存器SFR, Direct Addressing used to access SFRs

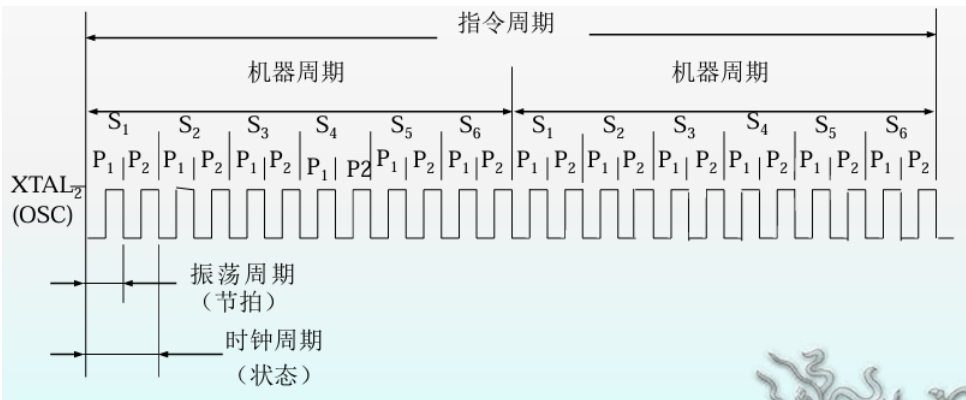
具体的一些汇编指令**应该不考**: MOV移动, DJNZ条件跳转, LJMP长跳转, SJMP短跳转, LCALL, ACALL (应该和地址有关吧?)

MOV的操作数#**十六进制**表示数据写入, **寄存器**表示数据加载

```
MOV A, #55H      ;load A with 55H
MOV P1, A        ;send 55H to p1
```

2.6 Timing (重要)

都是12兆晶振, the frequency of the crystal oscillator 晶振频率



oscillator periods振荡周期: 1/12 μs

machine cycle机器周期: 1μs

指令周期: 单周期指令; 双周期指令; 四周期指令

In original 8051, one machine cycle lasts 12 oscillator periods

CPU executing an instruction takes a certain number of clock cycles .These are referred as to as machine cycles(机器周期定义)

2.7 I/O Ports Configuration管脚

一共40个

P3系列比较重要，P3有8位各位都有additional function第二功能，一个功能占2位，共有四个功能。

P3 Bit	Function	Pin	
P3.0	RxD	10	Serial communications
P3.1	TxD	11	
P3.2	INT0	12	External interrupts
P3.3	INT1	13	
P3.4	T0	14	Timers
P3.5	T1	15	
P3.6	WR	16	Read/Write signals of external memories
P3.7	RD	17	

In systems based on 8751, 89C51 or DS89C4x0, pins 3.6 and 3.7 are used for I/O while the rest of the pins in port 3 are normally used in the alternate function role

自上而下为串行通讯，外部中断，外部计数器，读写功能

2.8 else

位寻址I/O Ports and Bit Addressability, 不需要记

unsigned number的加法

带进位的加法

减法，乘法（知道乘法比较慢），除法

都不需要记，因为都是讲的汇编指令的使用方法

2.9 C programming

unsigned char 8位最常见

sbit定义特殊功能位（管脚）

sbit a=P2^4;

代码基本框架

```
#include <reg51.h>
void main(void)
{
    unsigned char mynum[]="012345ABCD";
    unsigned char z;
    for (z=0;z<=10;z++)
        P1=mynum[z];
}
```

毫秒延迟的函数框架：

```
void MSDelay(unsigned int itime)
{
    unsigned int i,j;
    for (i=0;i<itime;i++)
        for (j=0;j<1275;j++);
}
```

这里的1275是针对于11.0592MHZ晶振算的，如果用12MHZ晶振， $12/11.0592*1275=1383.46$ ，可以取1383。

控制管脚高低电平直接令定义的管脚对应的变量为1或0

读位可寻址RAM，用bit，也就是位寄存器区域的数据（不重要）

```
#include <reg51.h>
sbit inbit=P1^0;
sbit outbit=P2^7;
bit membit; //use bit to declare
//bit- addressable memory
void main(void)
{
    while (1)
    {
        membit=inbit; //get a bit from P1.0
        outbit=membit; //send it to P2.7
    }
}
```

We use bit data type to access data in a bit-addressable section of the data RAM space 20 – 2FH

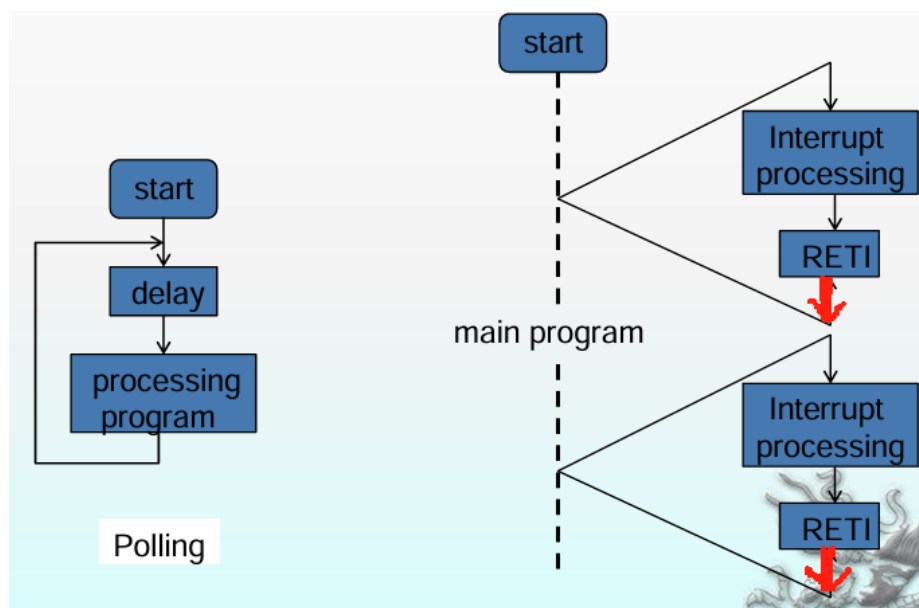
c语言的与或非，位运算（加上右移>>和左移<<）

		AND	OR	EX-OR	Inverter
A	B	A&B	A B	A^B	~B
0	0	0	0	0	1
0	1	0	1	1	0
1	0	0	1	1	
1	1	1	1	0	

ascii码和数字转换，首先将其每位分开，（120: 120/100, 120/10%10, 120%10）

通过串口发送需要对每一位转为ascii码：1+'0' 或者 1+0x30 或者 1 | 0x30

2.10 Interrupt中断



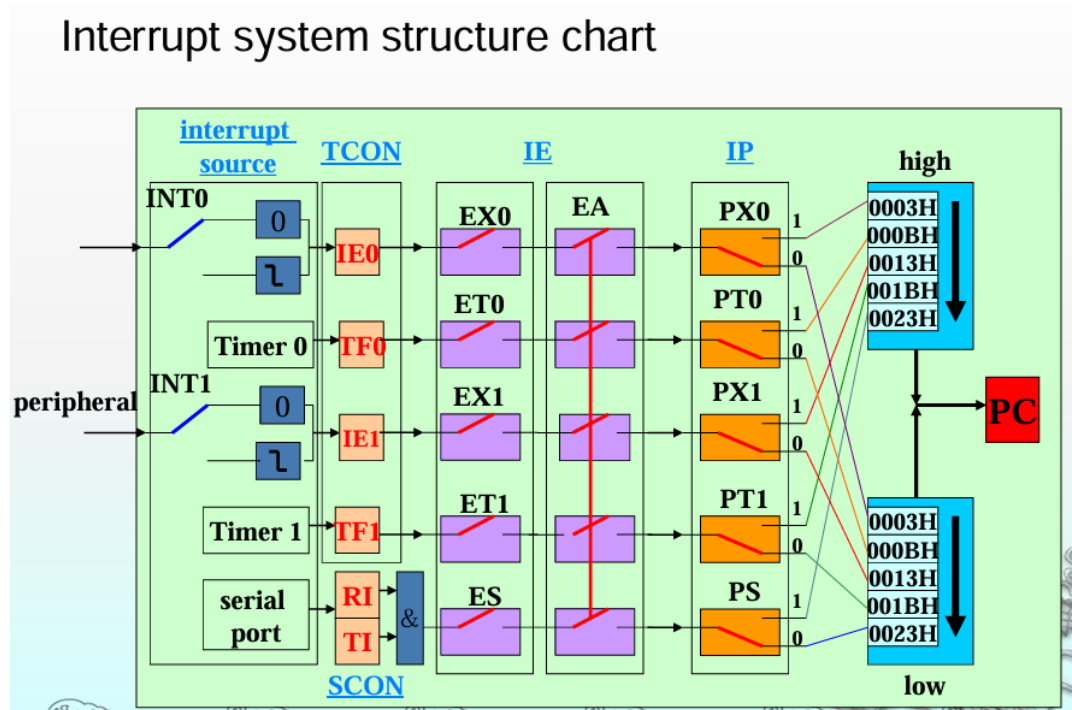
查询和中断

RETI是中断返回

2.10.1 中断的步骤Steps in Executing an Interrupt

1. It finishes the instruction it is executing and saves the address of the next instruction (PC) on the stack PC保留堆栈
2. It also saves the current status of all the interrupts internally (i.e: not on the stack) 其他状态值压入堆栈
3. It jumps to a fixed location in memory, called the interrupt vector table, that holds the address of the ISR 跳转到固定内存位置，被称为中断向量表，包含ISR的地址
4. The microcontroller gets the address of the ISR from the interrupt vector table and jumps to it. It starts to execute the interrupt service subroutine until it reaches the last instruction of the subroutine which is RETI 跳转到固定入口地址执行中断响应程序
5. Upon executing the RETI instruction, the microcontroller returns to the place where it was interrupted 执行RETI返回中断的位置
 - It gets the program counter (PC) address from the stack by popping the top two bytes of the stack into the PC 通过从堆栈中弹出前两个字节来获得PC Then it starts to execute from that address

2.10.2 interrupt system structure chart



中断所有关键字：

中断源：外部中断0-定时器0；外部中断1-定时器1；串口

中断标志：IE0 TF0 IE1 TF1 RI TI

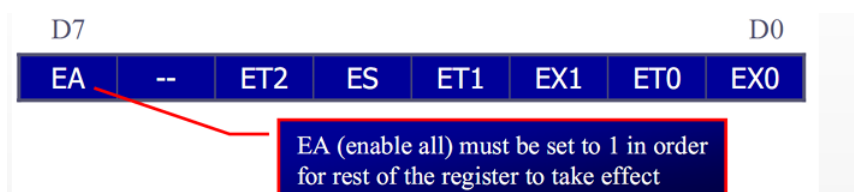
中断开关IE：EX。。。总开关EA

中断优先级

执行顺序默认是从地址地位向高位，即自上而下，写成二进制就是从右向左

2.10.3 IE interrupt enable 中断开关

具体对应关系参考上面的chart



ET2 : Enables or disables timer 2 overflow or capture interrupt (8052)

EA=1:中断使能打开

EA	IE.7	Disables all interrupts
--	IE.6	Not implemented, reserved for future use
ET2	IE.5	Enables or disables timer 2 overflow or capture interrupt (8952)
ES	IE.4	Enables or disables the serial port interrupt
ET1	IE.3	Enables or disables timer 1 overflow interrupt
EX1	IE.2	Enables or disables external interrupt 1
ET0	IE.1	Enables or disables timer 0 overflow interrupt
EX0	IE.0	Enables or disables external interrupt 0

2.10.4 中断向量表Interrupt vector table 不重要

表示了各个中断跳转的地址

Interrupt	ROM Location (hex)	Pin
Reset	0000	9
External HW (INT0)	0003	P3.2 (12)
Timer 0 (TF0)	000B	
External HW (INT1)	0013	P3.3 (13)
Timer 1 (TF1)	001B	
Serial COM (RI and TI)	0023	

2.10.5 IP interrupt priority 中断优先级

对应关系也参考上面的chart

interrupt priority register 中断优先级寄存器 bit-addressable

D7				D0			
--	--	PT2	PS	PT1	PX1	PT0	PX0

PT2 Timer 2 interrupt priority bit (8052 only)

--	IP.6	Reserved
--	IP.7	Reserved
PT2	IP.5	Timer 2 interrupt priority bit (8052 only)
PS	IP.4	Serial port interrupt priority bit
PT1	IP.3	Timer 1 interrupt priority bit
PX1	IP.2	External interrupt 1 priority bit
PT0	IP.1	Timer 0 interrupt priority bit
PX0	IP.0	External interrupt 0 priority bit

Priority bit=1 assigns high priority
Priority bit=0 assigns low priority

例子：

IE: 10001101

开了三个中断依次为：外部中断0，外部中断1，定时器1

IP: 00001010

首先看优先级的对应关系，按照地址从小到大的顺序，即从右向左，第一个有优先级的对应定时器0，但是他没开，第二个有优先级的是定时器1，因此首先执行定时器1；然后回看IE，依旧按照地址从小到大的顺序，先执行外部中断0，再外部中断1。那么最终的顺序为定时器1 外部中断0 外部中断1

2.10.6 c语言的中断响应

c语言中定义中断响应时，要在定义的函数后面加上关键字

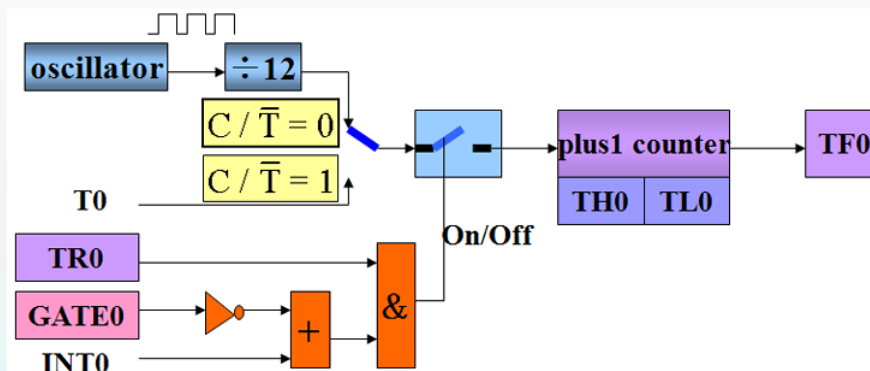
两个按键分别控制LED灯的开关, P3.2接口的按键按下时开灯, P3.3接口的按键按下时关灯。

```
void extern0() interrupt 0{}
void timer0() interrupt 1 {}
void extern1() interrupt 2{}
void timer1() interrupt 3 {}
void serial0() interrupt 4 {}
```

```
#include <reg51.h>
sbit LED = P1 ^ 0;
void INT_init (void){
    EA = 1;
    EX1 = 1;
    EX0 = 1;
    IT1 = 1; //1: falling edge-triggered
    IT0 = 1;
}
void INT_1 (void) interrupt 2 //using 2
{
    LED = 1; // turn off the light
}
void INT_0 (void) interrupt 0 // using 0
{
    LED = 0; //turn the light on
}
void main(void){
    INT_init(); //extern interrupt initialization
    while(1){
        //other program
    }
}
```

2.11 timer 定时器

Structure of Timer0



GATE0不考; TR0控制开关是否开, 取0时不计数; 计数来源可以是十二兆晶振, 记一次 $1\mu s$, 也可以来自于T0计数; 在plus1 counter处进行计数, 是16位计数, TH0是高八位, TL0是低八位, 当其计数到FFFF后TF0等于1, 计时结束。

算TH0 TL0, 对12MHZ晶振计一次为 $1\mu s$, 十六位最大为65536, 假设计时3ms即3000 μs , $65536 - 3000 = 62536$, 再转换为16进制分配给TH0 TL0。相当于初值的计算。

设置字节不考

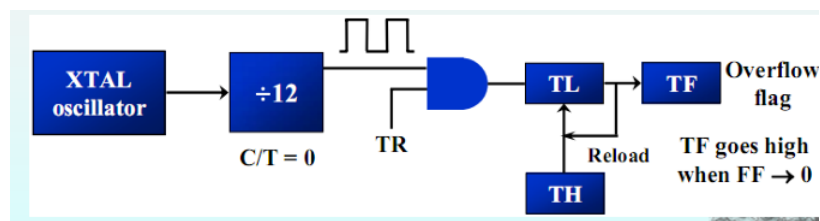
定时器只用模式1或2

M1	M0	Mode	Operating Mode
0	0	0	13-bit timer mode 8-bit timer/counter THx with TLx as 5-bit prescaler
0	1	1	16-bit timer mode 16-bit timer/counter THx and TLx are cascaded; there is no prescaler
1	0	2	8-bit auto reload 8-bit auto reload timer/counter; THx holds a value which is to be reloaded TLx each time it overflows
1	1	3	Split timer mode

模式1就是前面讲的：步骤为：（不是很重要）

1. Load the TMOD value register indicating which timer (timer 0 or timer 1) is to be used and which timer mode (0 or 1) is selected 先加载TMOD
2. Load registers TL and TH with initial count value 加载TH0 TL0
3. Start the timer
4. Keep monitoring the timer flag (TF) with the JNB TFx, target instruction to see if it is raised. Get out of the loop when TF becomes high 持续监控TF 直到TF溢出
5. Stop the timer 停止定时器
6. Clear the TF flag for the next round 清除TF标志
7. Go back to Step 2 to load TH and TL again 回到step2 重置

模式2 使用的是八位计数



在TH中存储初值，溢出后值给TL，当TL在256时溢出，TF=1；然后TF清除标志，初始值重新回到TH。

相较于模式1，在step4处

When the TL register rolls from FFH to 0 and TF is set to 1, TL is reloaded automatically with the original value kept by the TH register

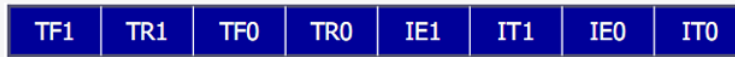
To repeat the process, we must simply clear TF and let it go without any need by the programmer to reload the original value

这使得模式2可以自动重新加载

TCON 不用全记住

◆ TCON (timer control) register is an 8-bit register

TCON: Timer/Counter Control Register



The upper four bits are used to store the TF and TR bits of both timer 0 and 1

The lower 4 bits are set aside for controlling the interrupt bits

只需要记住TF1 TR1 TF0 TR0

```
#include <reg51.h>
void T0Delay(void);
void main(void){
    while (1) {
        P1=0x55;
        T0Delay();
        P1=0xAA;
        T0Delay();
    }
}

void T0Delay(){
    TMOD=0x01;
    TL0=0x00;
    TH0=0x35;
    TR0=1;
    while (TF0==0);
    TR0=0;
    TF0=0;
}
```

典型例子，这里的TH0与TL0都是11.0592MHZ求的，一般用12MHZ。

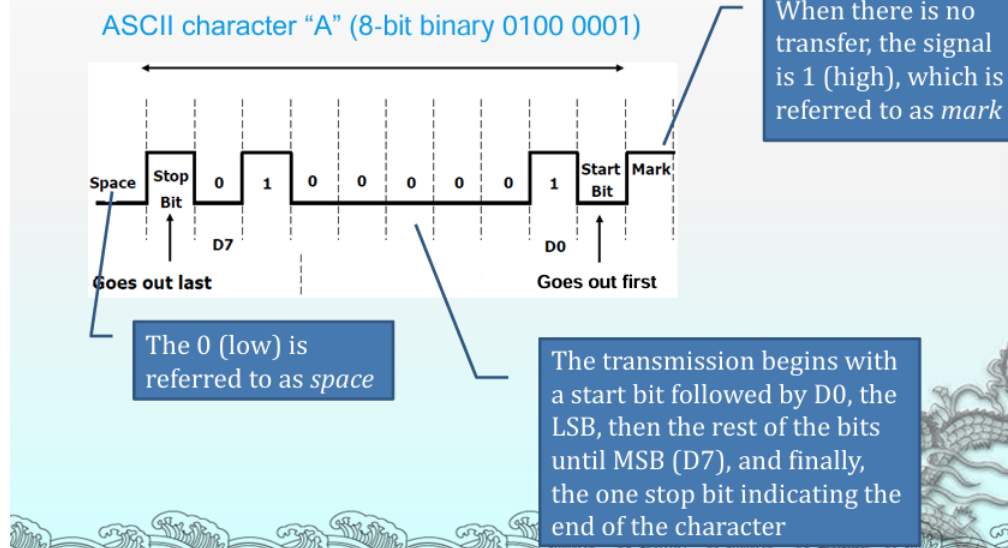
对右侧定时器每行的注释是：

- 初始化为模式1
- 加载初值
- TR0=1 启动定时器
- while 等待定时结束
- TR0=0 停止定时器
- TF0=1 清除定时标志

和上面的工作流程是完全对应的

2.12 serial communication 串口通讯

- ▶ The start bit is always a 0 (low) and the stop bit(s) is 1 (high)



这个图不考，串行通讯是一位一位发出的，平时是高电平，首个低电平标志开始，是起始位。发送的时候先低位后高位。

为什么要有波特率：To allow data transfer between the PC and an 8051 system without any error, we must make sure that the baud rate of 8051 system matches the baud rate of the PC's COM port 双方要有相同的频率来解析发送的波

当进行串口通讯时，为了解析不同值，所用的定时器的TH1也相应的不同，以MOD2工作时，baud rate与TH1的对照为，背下来就行

9600 ~ -3 FD

19200 ~ -3 FD

57600 ~ -1 最高频率溢出，初值为FF

28800 ~ -2

SBUF register

发送和接收的缓存，但是有两个不同的缓存器

SCON 不需要记

SCON的RI TI 是中断标志

串行通讯工作过程：

发送数据

1. TMOD register is loaded with the value 20H, indicating the use of timer1 in mode 2 (8-bit auto-reload) to set baud rate 设置模式2自动可重载
2. The TH1 is loaded with one of the values to set baud rate for serial data transfer 算波特率
3. The SCON register is loaded with the value 50H, indicating serial mode1, where an 8-bit data is framed with start and stop bits 设置串行通讯模式中断

4. TR1 is set to 1 to start timer 开定时器，启动波特率发生器
5. TI is cleared by CLR TI instruction
6. The character byte to be transferred serially is written into SBUF register
7. The TI flag bit is monitored with the use of instruction JNB TI , xx to see if the character has been transferred completely
8. To transfer the next byte, go to step 5

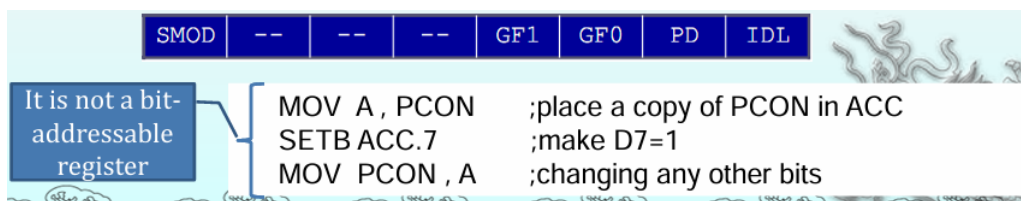
TI代表发送一个数，先清零，发送一个数，发送完成后TI变成1，可以进行下一次发送（一个用来标记发送成功的flag）

RI表示接收一个数，接收成功RI=1，但是RI什么时候能够触发取决于发送端（抽象判断题）

接收数据：

1. TMOD register is loaded with the value 20H, indicating the use of timer 1 in mode2 (8-bit auto-reload) to set baud rate
2. TH1 is loaded to set baud rate
3. The SCON register is loaded with the value 50H, indicating serial mode 1, where an 8-bit data is framed with start and stop bits
4. TR1 is set to 1 to start timer 1
5. RI is cleared by CLR RI instruction
6. The RI flag bit is monitored with the use of instruction JNB RI,xx to see if an entire character has been received yet
7. When RI is raised, SBUF has the byte, its contents are moved into a safe place
8. To receive the next character, go to step 5

波特率加倍：



有一个SMOD可以标定波特率是否加倍（不重要），比如9600不加倍是-3，加倍是-6 对于TH1来说

Example 11-7

Write a C program for 8051 to transfer the letter “A” serially at 4800 baud continuously. Use 8-bit data and 1 stop bit.

Solution:

```
#include <reg51.h>
void main(void){
    TMOD=0x20; //use Timer 1, mode 2
    TH1=0xFA; //4800 baud rate
    SCON=0x50;
    TR1=1;
    while (1) {
        SBUF='A'; //place value in buffer
        while (TI==0);
        TI=0;
    }
}
```

这个程序展示了串行通讯发送过程，其流程和上述讲过的一致。TR1表示打开定时器，TI是发送成功的flag

这里SBUF='A'，表示串行通讯发送一个A

如果是ACC=SBUF，表示串行通讯接收值

2.13 interface

LCD ADC KEY

2.13.1 矩阵按键

按键识别方法：行扫描和线路反转法

行扫描：

设置行线为输出，列线为输入，当无按键按下时，列输入全为“1”。设计时，将某一行输出为“0”，读取列线值，若其中某一位为“0”，则表明行、列交叉点处的按键被按下，否则无按键按下；继续扫描下一行（将下一行输出为“0”），直至全扫描完为止。

或同理可设置列线为输出，行线为输入。设计时，将某一列输出为“0”。

线路反转法：需要两个双向I/O口分别接行、列线

1. 由行线输出全“0”，读入列线，判有无键按下（若有一个列为“0”则表明有键按下）。
2. 若有键按下,再将读入的列值从列线输出,读取行线的值。
3. 第一步读入的列值与第二步读入的行值运算，从而得到代表此键的唯一的特征值。

判断速度快，两次即可

```

unsigned char Keyboard (void)
{
    unsigned char Rank, Row;

    P0 = 0xf0;           //置所有行线为低电平
    if(P0 != 0xf0)       //判断所有列线是否全为高电平, 若全为高, 则无键按下
    {
        delay();         //软件延时消抖
        if(P0 != 0xf0)
        {
            P0 = 0xf0;   //行线输出低电平, 列线作输入
            if(P0_4==0)   Rank=0;   //判断被按下的按键所处的列线
            else if(P0_5==0) Rank =1;
            else if(P0_6==0) Rank =2;
            else if(P0_7==0) Rank =3;

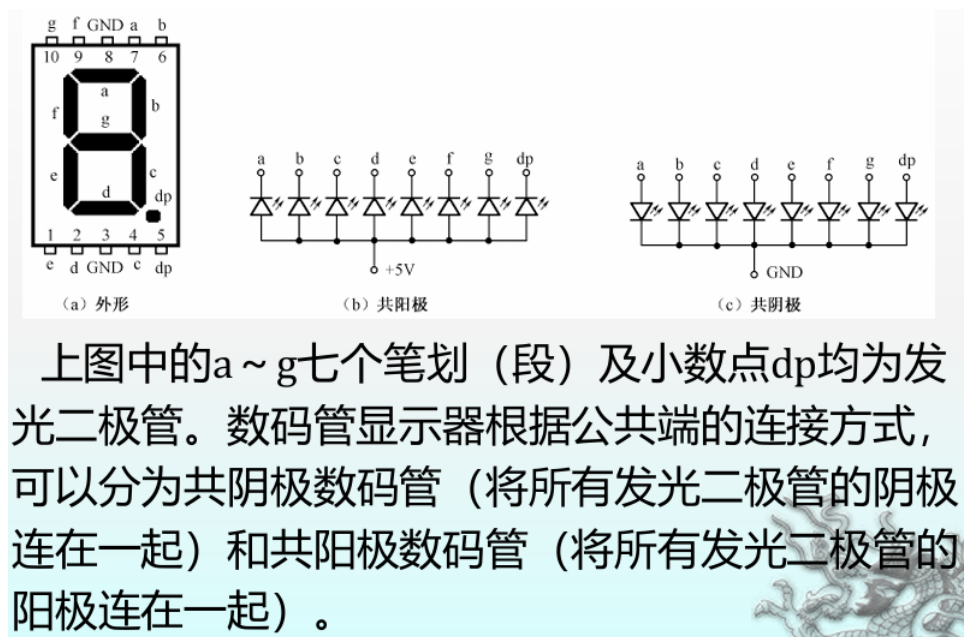
            P0 = 0x0f;    //列线输出低电平, 行线作输入
            if(P0_0==0)   Row =0;   //判断被按下的按键所处的行线
            else if(P0_1==0) Row =1;
            else if(P0_2==0) Row =2;
            else if(P0_3==0) Row =3;

            return (Rank + Row *4)   //返回按键的键值
        }
    }
}

```

2.14.2 led显示

和前面一样



上图中的a~g七个笔划（段）及小数点dp均为发光二极管。数码管显示器根据公共端的连接方式，可以分为共阴极数码管（将所有发光二极管的阴极连在一起）和共阳极数码管（将所有发光二极管的阳极连在一起）。

2.14.3 lcd1602

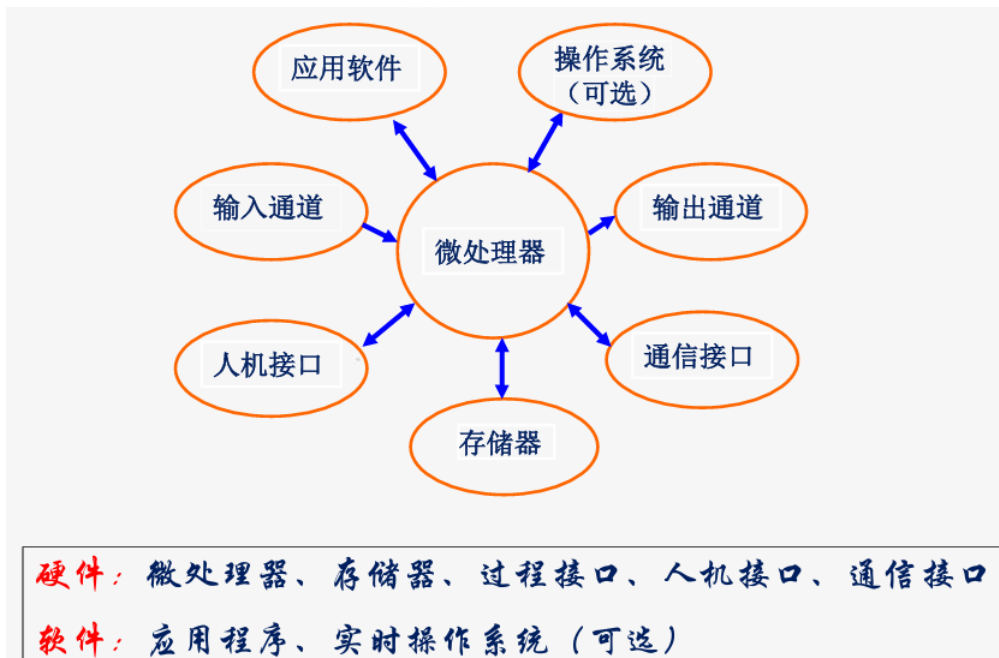
不考，复习个毛

3 嵌入式系统

3.1 定义

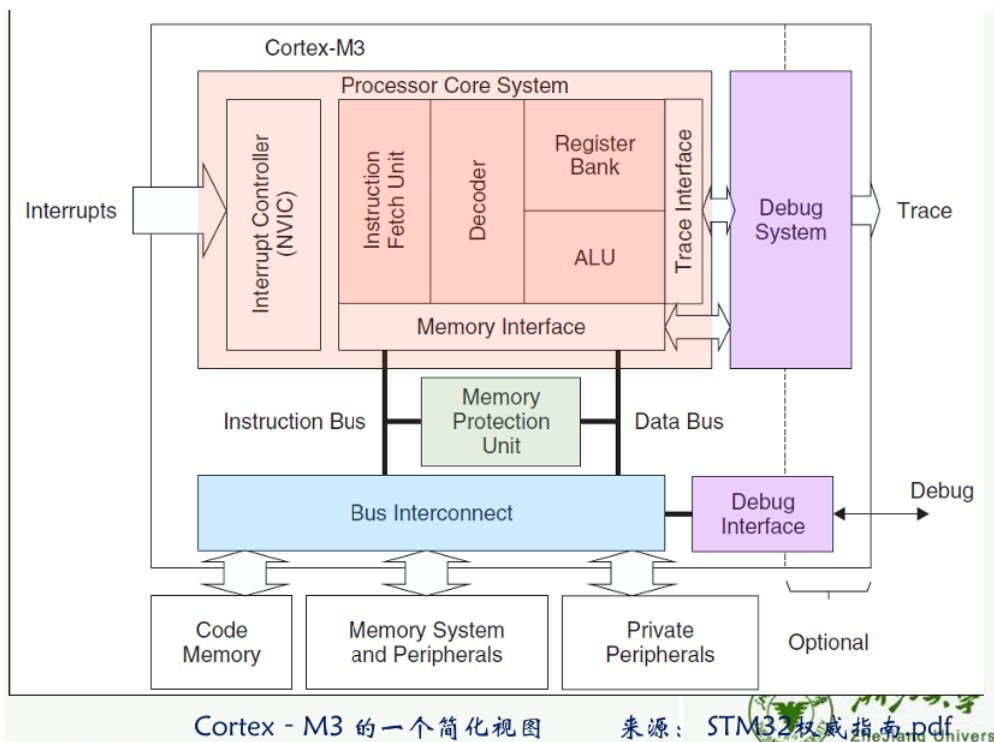
以应用为中心、以计算机技术为基础，软硬件可裁减，适用于应用系统的、对功能、可靠性、成本、体积、功耗有严格要求的专用计算机系统。

组成模型：



冯诺依曼的体系结构只是嵌入式系统里面的某一个芯片的结构

3.2 cortex M3

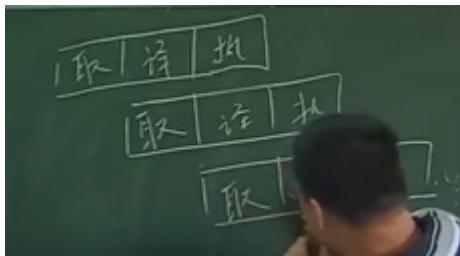


这个图记下来，最多简答题

相较于8051：有一个复杂的中断，有三级流水线，有debug系统，多了内存保护单元

三级流水线：取指令 译码 执行

三个步骤占用的资源都不一样，三者时间上是错开的



正常操作过程中，在执行一条指令的同时 对下一条指令进行译码，并将第三条指令从存 储器中取出。

流水线上各指令的地址		流水线工 位	描述
ARM指令 集	Thumb指令 集		
PC	PC	取指	指令从存储器中取出
PC-4	PC-2	译码	对指令使用的寄存器进行译码
PC-8	PC-4	执行	从寄存器组中读出寄存器，执行 移位和ALU操作，寄存器被写回 到寄存器组中

thumb-2 相较于原来的内核性能没有衰减，但是代码容量只有之前的3/4，效率更高

3.3 两种工作模式

1) 线程模式和处理模式

目的：引入处理模式（handle mode）和线程模式（thead mode）的本意，是用于区别普通应用程序（线程模式）和中断服务程序（处理模式）；

效果：有效的简化了ARM7体系结构支持7种处理器 模式(用户模式、快中断模式、中断模式、管理模式、中止模式、未定义模式和系统模式)。

2) 特权级和用户级：

目的：用于存储器访问的保护机制。使得普通的用户程序代码 不能意外地，甚至是恶意地执行涉及到要害的操作；

特权级：该级别的程序可以访问所有范围的存储器（如果有 MPU，还要在MPU 规定的禁地之外），并且可以执行所有指 令；

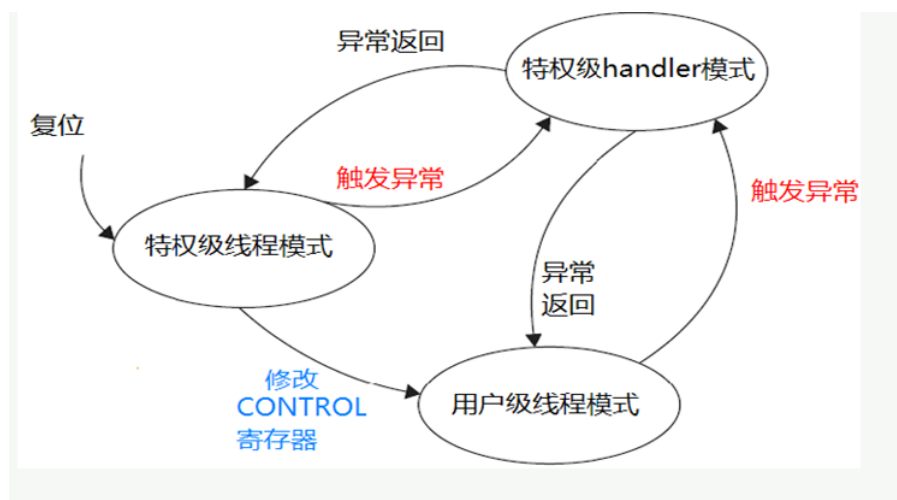
用户级：用户级程序不能直接改写CONTROL 寄存器，需执行 一条系统调用指令(SVC)，由异常服务例程修改CONTROL 寄 存器，才能在用户级的线程模式下重新进入特权级。

3) 默认状态：

在CM3 运行主程序（后台程序）时是线程模式，既可以使用特权级，也可以使用用户级；

中断(异常)服务程序必须在处理模式（始终特 权级）下执行；

复位后，mcu默认进入线程模式，特权极访问。



(看着这么多，其实最多考个选择题)

3.3 存储器保护单元MPU

stm32不允许随意改内存值，改sfr值，只能通过函数调用来改

3.4 内部寄存器

1) 组成：CM3 处理器拥有R0 - R15 的寄存器组。其中R13 作为堆栈指针SP，SP 有两个，但在同一时刻只能有一个可以使用。

R0	通用寄存器	Low Registers
R1	通用寄存器	
R2	通用寄存器	
R3	通用寄存器	
R4	通用寄存器	
R5	通用寄存器	
R6	通用寄存器	
R7	通用寄存器	
R8	通用寄存器	High Registers
R9	通用寄存器	
R10	通用寄存器	
R11	通用寄存器	
R12	通用寄存器	
R13 (MSP)	R13 (PSP)	主堆栈指针(MSP)，进程堆栈指针(PSP)
R14		连接寄存器(LR)
R15		程序计数器(PC)

记住下面三个，最多考个选择题

3.5 中断机制

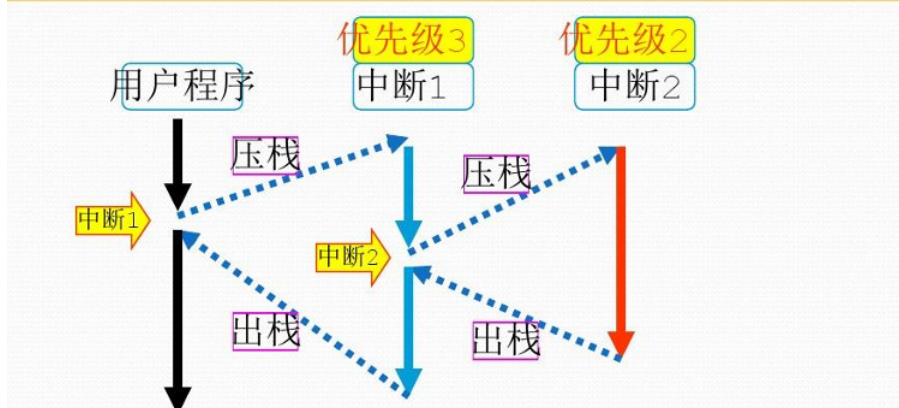
中断优先级数目有八位 256级

中断可配置的数目有240个

stm为了加速中断有占先和末尾连锁

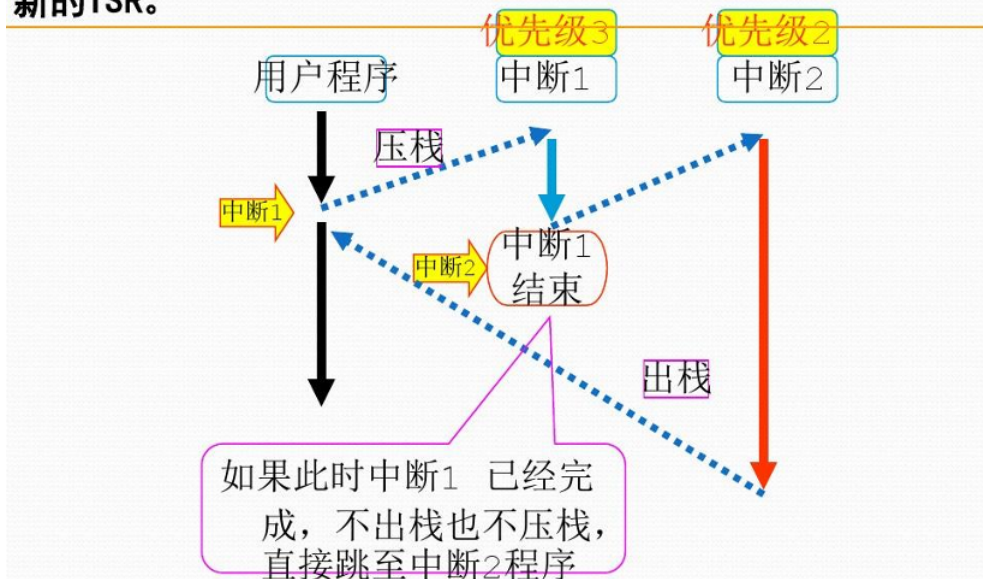
占先：优先级打断

在异常处理程序中，一个新的异常比当前的异常优先级更高，处理器打断当前的流程，响应优先级更高的异常，此时产生中断嵌套。



末尾连锁：

末尾连锁是处理器用来加速中断响应的一种机制。在结束ISR时，如果存在一个挂起中断，其优先级高于正在返回的ISR或线程，那么就会跳过出栈操作，转而将控制权让给新的ISR。



一个中断结束后不像8051后会到原位出栈再堆栈，而是直接跳转到另一个中断，最后一起出栈。

3.6 存储器

stm32 是32位的，因此

一个字节8位，一个字有32位，即4个字节。一个半字有16位

存储器格式从来不考

CM3处理器两种调试接口（没考过，但是也记一下）

JTAG: 6线制接口

SWD: 2线制接口

3.7 功能部件

读代码

3.7.1 GPIO

① DI 数字量输入：高电平1，低电平0，输入中断、唤醒等；

② DO 数字量输出：高电平1，低电平0；

③ AF 其它功能：引脚复用，预定义的功能模块引脚；

工作模式：

1) input floating：浮空输入，引脚电平不确定

input pull-up：上拉输入，引脚高电平，输入低电平时起作用

input pull-down：下拉输入，引脚低电平，输入高电平时起作用

2) output push-pull：推挽输出，输出非0即1

output open-drain：开漏输出，不附加上拉电阻时没有高电平输出能力

3) Alternate function push-pull、open-drain：交替功能式输出

应该考不了这么细

七、应用例

```
void LED_Sets(uint8_t data)
{
    uint16_t setValue;

    setValue = GPIO_ReadOutputData(GPIO_LED); //调用API
    //((uint16_t)GPIOx->ODR)-->setValue

    setValue &= 0x00ff;
    setValue |= (uint16_t)data << 8;
    //data-->setValue

    GPIO_Write(GPIO_LED, setValue);           //调用API
    //setValue-->((uint16_t)GPIOx->ODR)       //等效的寄存器操作
}
```

记住函数的读写用这两个函数，不会让你写，最多标注释

3.7.2 定时器

数目更多：4个通用16位定时器

模式更多：上升、下降、上升和下降

T1 为增强型，带互补输出，紧急停止等功能，可用于电机控制

中断产生条件：定时器溢出、比较值相等、捕获引脚有指定的跳变

```

void TIM5_IRQHandler(void)
{
    /* www.armjishu.com ARM技术论坛 */
    static u32 counter = 0;

    if (TIM_GetITStatus(TIM5, TIM_IT_Update) != RESET)
    {
        TIM_ClearITPendingBit(TIM5, TIM_IT_Update);
        /* LED1指示灯状态取反 */
        SZ_STM32_LEDToggle(LED1);

        /* armjishu.com提醒您: 不建议在中断中使用Printf, 此示例只是演示。 */
        printf("\n\rarmjishu.com提醒您: 不建议在中断中使用Printf, 此示例只是演示。 \n\r");
        printf("ARMJISHU.COM-->TIM5:%d\n\r", counter++);
    }
}

```

记住清楚中断标志，读中断状态的函数，

3.7.3 串行通讯usart

和8051一样，都是异步的，波特率也一样，只是更快一些

工作流程：

接收过程：UART监听总线，有下跳变时，启动 数据采样，一个字节的的数据收到后，如果奇偶校 验正确，则把数据存到接收寄存器中，置状接收 态标志位，并向CPU申请中断，让CPU及时读取；

发送过程：UART的发送寄存器接收到CPU写入 的数据，立刻启动，按设定的参数逐位发送，发 送完毕，置状发送态标志位，并向CPU申请中断，告诉CPU可以发送下一个字节。

和8051大致相同

```

void USART1_IRQHandler(void)
{
    u8 k;
    if (USART_GetITStatus(USART1, USART_IT_RXNE) != RESET) //检查指定的USART中断发生与否
    {
        k = USART_ReceiveData(USART1);
        k++;
        USART_SendData(USART1, k); //通过外设USARTx发送单个数据
        //USART_ReceiveData(USART1) 返回USARTx最近接收到的数据
        while (USART_GetFlagStatus(USART1, USART_FLAG_TXE) == RESET);
    }
}

```

检查中断状态，接收串行通讯数据，发送串行通讯数据，判断是否发送完成

```

int main()
{
    RCCINIT(); //系统时钟的初始化
    GPIOINIT(); // 端口的初始化
    USARTINIT(); // 串口的配置及其初始化
    NVICINIT(); // 中断模式的初始化
    while(1);
}

```

具体怎么初始化的不用记，知道叫什么名字对应哪个模块就行

3.7.4 A/D 必考

ADDA数值转换，线性关系：

对于普通的12位，最大数字值为4096。已知Vref=2.5V，Vin=0.75V， $Vin/Vref = D \text{ 数字值} / 4096$

或者反之用数字值求电压值

读代码：


```

int main()
{
    u32 ad=0;
    u8 i;
    RCCINIT_PRINTF(); //初始化printf的系统时钟
    RCCINIT_ADC(); //初始化ADC的系统时钟
    GPIOINIT_ADC(); //初始化ADC的端口配置
    GPIOINIT_PRINTF();
    USARTINIT_PRINTF(); //printf串口的初始化配置
    NVICINIT_PRINTF(); //printf中断模式的初始化配置
    ADCINIT_ADC();
    while(1)
    {
        ad=0;
        for(i=0;i<50;i++)//读取50次的AD数值取其平均数较为准确
        {
            ADC_SoftwareStartConvCmd(ADC1, ENABLE);
            while(!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC)); //转换结束标志位
            ad=ad+ADC_GetConversionValue(ADC1); //返回最近一次ADCx规则组的转换结果
        }
        ad=ad/50;
        printf("ad=%f\n", ad*3.3/4096);
        delay_ms(1000);
    }
}

```

AD软件触发启动，等待转换结束，得到转换的值。连续做50次转换求均值（难度接近考试？）

3.7.5 D/A 必考

与AD类似，一般0-2.5V 也可能3.3V，看具体题目

```

int main()
{
    u8 i;
    float da;
    RCCINIT();
    GPIOINIT();
    NVICINIT();
    USARTINIT();
    DACINIT();
    while(1)
    {
        da=0;
        for(i=0;i<=10;i++)
        {
            da=i*400;
            //12位 右对齐 PA4 端口输出
            DAC_SetChannel1Data(DAC_Align_12b_R, da);
            printf("da=%fv\n", 3.3*da/4096);
            delayms(1000);
            delayms(1000);
            delayms(1000);
            delayms(1000);
            delayms(1000); //间隔5秒输出一个电压
        }
    }
}

```

可能会让你画输出的波形，如阶梯波

DA输出最大是4096



2025/6/3 11:51 | 811 字 | 默认笔记本

标题

判断10

选择15

简答4 times 5

程序 填空 阅读 注释 20

编程题

综合题

与非 或非 异或

门电路：能够用半导体元器件表示一个逻辑电路

组合逻辑电路与时序逻辑电路定义，举例

数字电路：简单的模块构建复杂的功能

时序：有记忆，输出与过去有关

组合：非时序

会举例就行，知道是干什么的，不用知道内部结构，不用仔细看(组合编码器，译码器)(时序计数器)

数码管：共阴共阳

触发器：记住个D触发器，别的没考过

冯诺依曼计算机结构

汇编有哪些构成

单片机内部三总线 地址 数据 控制bus

PSW是什么(状态寄存器)，每一位是什么

堆栈：选择题，push先指针加再加数据，pop先出数据再指针减

PC：

内部RAM分区，不会很细

汇编记个名字

timing，十二兆晶振，震荡周期1/12微秒，机器周期1微秒，指令周期

8051 P3管脚，第二功能，八位四个功能

c语言程序框架

毫秒延时器

sbit定义管脚

ascii码

中断：概念 RETI

几个步骤

外部中断0定时器0那一页记住

优先级从0到1

定时器0算时间 TC TL

初始值怎么算

tmode只考虑12

TR0TR1TF0TF1记住

example10-17记住

数据传递有起始位，第一个低电平表示开始，结束后回到高电平

串行通讯每发八位就要加一个起始位和结束位

发射和接受要有相同的频率

9600-3or-6

19200-3

28800-2

57600-1

串行通讯如何工作

example 11-7程序

矩阵按键

翻转法程序

led显示

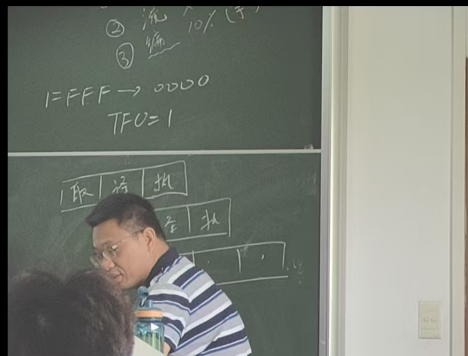
lcd不考

嵌入式系统定义 记下面的一个

嵌入式组成模型

经典的内部结构图：cortex m3

三级流水线 取指令 译码 执行



thumb2 代码量变3/4但是性能不变

stm32两种工作模式

线程模式和处理模式

线程模式下分

什么是MPU

R13 R14 R15通用寄存器

中断特点 两个数字

占先，中断嵌套

存储器，32位 字节半字

接口

stm32读代码

GPIO：怎么读写一个管脚两个函数

中断 定时器

usart 串行通讯 程序例六 一些经典模块初始化

AD转换

Vref与总数字比例关系

10位是1024

12位是4096

